University of Alicante

Department of Computer Science and Artificial Intelligence

Computer Engineering – Specialisation in Computing

VAR: Computer Vision and Robotics

3D Gaussian Splatting

Mathematical Foundations and Rendering Pipeline

A Comparative Study with Neural Radiance Fields

Author: [Author Name]
Date: March 2026

Contents

1 Introduction and State of the Art **3**
- 1.1 Novel View Synthesis . . . 3
- 1.2 Neural Radiance Fields (NeRF) . . . 3
- 1.3 3D Gaussian Splatting (3DGS) . . . 3
- 1.4 Comparative Analysis: NeRF vs. 3DGS . . . 4

2 Mathematical Representation of 3D Gaussians **4**
- 2.1 The 3D Gaussian Kernel . . . 4
- 2.2 Gaussian Primitive . . . 4
- 2.3 Why Gaussians? . . . 5
- 2.4 Scene Representation and Parameter Count . . . 5

3 Covariance Factorisation and Rotation **5**
- 3.1 The RS Factorisation . . . 5
- 3.2 Quaternion Representation of Rotation . . . 6
- 3.3 Scale Parametrisation . . . 6
- 3.4 Summary . . . 6

4 Colour Modelling via Spherical Harmonics **6**
- 4.1 Motivation . . . 6
- 4.2 View-Dependent Colour . . . 6
- 4.3 Degree Selection . . . 7

5 2D Projection and Jacobian Approximation **7**
- 5.1 Overview . . . 7
- 5.2 Camera Model and Mean Projection . . . 7
- 5.3 Covariance Projection via Jacobian Approximation . . . 7
- 5.4 Bounding Box and Tile Culling . . . 8

6 Differentiable Rasterisation Pipeline and Tiling **8**
- 6.1 Design Goals . . . 8
- 6.2 Pipeline Overview . . . 8
- 6.3 Tile Decomposition . . . 8
- 6.4 CUDA Rendering and Early Termination . . . 9
- 6.5 Backward Pass . . . 9

7 Alpha Blending and Accumulation Rendering **9**
- 7.1 Alpha Compositing . . . 9
- 7.2 Per-Gaussian Opacity at a Pixel . . . 9
- 7.3 Incremental Accumulation . . . 9
- 7.4 Connection to NeRF Volume Rendering . . . 10

8 Optimisation Mechanisms: Densification, Pruning, and Opacity **10**
- 8.1 Training Objective . . . 10
- 8.2 Optimiser . . . 10
- 8.3 Adaptive Density Control . . . 10
- 8.4 Pruning . . . 10
- 8.5 Periodic Opacity Reset . . . 11

8.6 Training Schedule and Convergence 11

1 Introduction and State of the Art

1.1 Novel View Synthesis

Novel view synthesis (NVS) aims to generate photorealistic images from arbitrary viewpoints given a set of images captured from known camera positions. Applications include virtual reality, autonomous driving, and digital heritage preservation. Deep learning has replaced traditional multi-view stereo methods with learned representations, and two have emerged as state-of-the-art: **NeRF** [4] and **3DGS** [3].

1.2 Neural Radiance Fields (NeRF)

NeRF represents a scene as a continuous volumetric function:

$$F_\theta : (\mathbf{x}, \mathbf{d}) \longrightarrow (\mathbf{c}, \sigma) \tag{1}$$

where F_θ is an MLP mapping position $\mathbf{x} \in \mathbb{R}^3$ and viewing direction $\mathbf{d}$ to colour $\mathbf{c}$ and density σ. A pixel colour is computed by integrating along the ray:

$$\hat{C}(\mathbf{r}) = \int_{t_n}^{t_f} T(t)\,\sigma(\mathbf{r}(t))\,\mathbf{c}(\mathbf{r}(t), \mathbf{d})\,\mathrm{d}t, \quad T(t) = \exp\left(-\int_{t_n}^{t} \sigma(\mathbf{r}(s))\,\mathrm{d}s\right) \tag{2}$$

Limitations. NeRF requires 1–2 days of training and renders at 5–30 s/frame due to the cost of per-pixel MLP evaluations.

1.3 3D Gaussian Splatting (3DGS)

3DGS represents the scene as a set of *explicit, learnable 3D Gaussian primitives.* Each Gaussian $\mathcal{G}_i$ stores a position $\boldsymbol{\mu}_i \in \mathbb{R}^3$, a covariance $\boldsymbol{\Sigma}_i$, an opacity o_i, and view-dependent colour as spherical harmonic (SH) coefficients. Rendering is performed by projecting each 3D Gaussian onto the image plane (splatting) and compositing the resulting 2D splats via differentiable alpha blending on the GPU, achieving >90 fps at 1080p. Gaussians are initialised from a SfM sparse point cloud and optimised end-to-end with adaptive density control (Section 8).

1.4 Comparative Analysis: NeRF vs. 3DGS

Table 1: Qualitative comparison between NeRF and 3DGS.

Criterion	**NeRF**	**3D Gaussian Splatting**
Representation	Implicit (MLP)	Explicit (Gaussian primitives)
Rendering method	Ray casting + volume integration	GPU rasterisation (splatting)
Render speed	<1–30 fps	>90 fps (consumer GPU)
Training time	1–2 days; 5 min (Instant-NGP)	20–40 minutes
Memory (inference)	Low (MLP weights)	High (∼1 GB/scene)
Editability	Difficult	Easy (explicit geometry)

NeRF's implicit representation handles smooth geometry naturally; 3DGS trades spatial continuity for hardware-accelerated rendering and explicit editability.

The remainder of this document covers the mathematical foundations of 3DGS: geometric representation (Sections 2–5), the rendering pipeline (Sections 6–7), and the optimisation strategy (Section 8).

2 Mathematical Representation of 3D Gaussians

2.1 The 3D Gaussian Kernel

A 3D Gaussian is defined by its unnormalised influence kernel:

$$\boxed{\mathcal{G}(\mathbf{x}\,;\,\boldsymbol{\mu},\boldsymbol{\Sigma}) = \exp\left(-\frac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right)} \tag{3}$$

where $\boldsymbol{\mu} \in \mathbb{R}^3$ is the centre and $\boldsymbol{\Sigma} \in \mathbb{R}^{3\times 3}$ is a symmetric positive semi-definite (SPSD) covariance matrix. The kernel equals 1 at $\mathbf{x} = \boldsymbol{\mu}$ and decays with the Mahalanobis distance; the locus $\mathcal{G} = e^{-1/2}$ traces the *one-sigma ellipsoid*.

2.2 Gaussian Primitive

Gaussian Primitive

Each primitive $\mathcal{G}_i$ is an oriented, anisotropic ellipsoid characterised by:

- $\boldsymbol{\mu}_i \in \mathbb{R}^3$ – **position**,
- $\boldsymbol{\Sigma}_i \in \mathbb{R}^{3\times 3}$ – **shape and orientation** (eigenvalues = squared semi-axis lengths; eigenvectors = principal axes),
- $o_i \in [0,1]$ – **opacity**,

- $\mathbf{f}_i \in \mathbb{R}^{(L+1)^2 \times 3}$ – **colour** as SH coefficients (Section 4).

2.3 Why Gaussians?

Desirable Properties

1. **Differentiability**: infinitely differentiable w.r.t. all parameters, enabling gradient-based optimisation.
2. **Closed-form projection**: a 3D Gaussian projects to a 2D Gaussian under a perspective camera (Section 5).
3. **Soft boundaries**: smooth fall-off models translucent or fuzzy regions (hair, smoke) without hard silhouettes.
4. **Efficient rasterisation**: 2D splats are rendered in a single GPU forward pass via a tile-based rasteriser.

2.4 Scene Representation and Parameter Count

A scene is a finite mixture of N primitives:

$$\mathcal{S} = \left\{(\boldsymbol{\mu}_i,\, \boldsymbol{\Sigma}_i,\, o_i,\, \mathbf{f}_i)\right\}_{i=1}^{N} \tag{4}$$

Typical scenes contain $N \in [10^5, 10^7]$ Gaussians stored as flat GPU tensors. For SH degree $L = 3$ each Gaussian has 59 learnable parameters $(3 + 4 + 3 + 1 + 48)$, so 3×10^6 Gaussians require ≈ 700 MB in `float32`.

Note

Directly optimising the 9 entries of $\boldsymbol{\Sigma}$ does not guarantee SPSD. 3DGS instead parametrises $\boldsymbol{\Sigma}$ through a factorisation that enforces this constraint by construction (Section 3).

3 Covariance Factorisation and Rotation

3.1 The RS Factorisation

To guarantee SPSD throughout optimisation, 3DGS parametrises $\boldsymbol{\Sigma}$ as:

$$\boxed{\boldsymbol{\Sigma} = \mathbf{R}\,\mathbf{S}\,\mathbf{S}^\top\mathbf{R}^\top} \tag{5}$$

where $\mathbf{R} \in SO(3)$ is a rotation matrix and $\mathbf{S} = \mathrm{diag}(s_1, s_2, s_3)$ with $s_k > 0$. SPSD is guaranteed since $\mathbf{v}^\top\boldsymbol{\Sigma}\,\mathbf{v} = \|\mathbf{S}^\top\mathbf{R}^\top\mathbf{v}\|^2 \geq 0$ for all $\mathbf{v}$. Expanded:

$$\boldsymbol{\Sigma} = \mathbf{R}\begin{pmatrix} s_1^2 & 0 & 0 \\ 0 & s_2^2 & 0 \\ 0 & 0 & s_3^2 \end{pmatrix}\mathbf{R}^\top \tag{6}$$

The columns of $\mathbf{R}$ are the eigenvectors of $\boldsymbol{\Sigma}$ and s_k^2 are the eigenvalues, so s_k are the principal semi-axis lengths.

3.2 Quaternion Representation of Rotation

$\mathbf{R}$ is stored as a unit quaternion $\mathbf{q} = (q_w, q_x, q_y, q_z)$, $\|\mathbf{q}\| = 1$, which uses only 4 parameters and avoids gimbal lock:

$$\mathbf{R}(\mathbf{q}) = \begin{pmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_wq_z) & 2(q_xq_z + q_wq_y) \\ 2(q_xq_y + q_wq_z) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_wq_x) \\ 2(q_xq_z - q_wq_y) & 2(q_yq_z + q_wq_x) & 1-2(q_x^2+q_y^2) \end{pmatrix} \tag{7}$$

After each gradient step $\mathbf{q}$ is renormalised: $\mathbf{q} \leftarrow \mathbf{q}/\|\mathbf{q}\|$.

3.3 Scale Parametrisation

Scales are stored as $\hat{s}_k = \log s_k \in \mathbb{R}$ (unconstrained) and recovered as $s_k = \exp(\hat{s}_k)$, ensuring $s_k > 0$ without constrained optimisation. Opacity is handled analogously via logit space.

3.4 Summary

Table 2: Learnable parameters per Gaussian primitive.

Attribute	**Symbol**	**Dim.**	**Constraint**
Position	$\boldsymbol{\mu}$	3	$\mathbb{R}^3$
Rotation	$\mathbf{q}$	4	$\|\mathbf{q}\| = 1$
Scale (log)	$\hat{\mathbf{s}}$	3	$\mathbb{R}^3$
Opacity (logit)	$\hat{o}$	1	$\sigma(\hat{o}) \in (0,1)$
SH coefficients	$\mathbf{f}$	$3(L+1)^2$	$\mathbb{R}$

4 Colour Modelling via Spherical Harmonics

4.1 Motivation

Surfaces exhibit view-dependent appearance (specular highlights, metallic reflections). In 3DGS each Gaussian encodes its view-dependent colour compactly using **spherical harmonics (SH)**, an orthonormal basis on the unit sphere S^2:

$$\int_{S^2} Y_l^m(\mathbf{d})\, Y_{l'}^{m'}(\mathbf{d})\, \mathrm{d}\mathbf{d} = \delta_{ll'}\, \delta_{mm'} \tag{8}$$

Low-degree terms capture diffuse colour; higher degrees encode specular effects.

4.2 View-Dependent Colour

For Gaussian $\mathcal{G}_i$ and channel $k \in \{R, G, B\}$, the colour in direction $\mathbf{d}$ is:

$$\boxed{c_{i,k}(\mathbf{d}) = \sum_{l=0}^{L} \sum_{m=-l}^{l} f_{i,k,l}^m\, Y_l^m(\mathbf{d})} \tag{9}$$

where $f^m_{i,k,l}$ are learned coefficients. In matrix form: $\mathbf{c}_i(\mathbf{d}) = \mathbf{F}_i\,\mathbf{y}(\mathbf{d})$, with $\mathbf{y}(\mathbf{d}) \in \mathbb{R}^{(L+1)^2}$ the vector of SH values. SH evaluation is purely polynomial, making it very efficient on the GPU.

4.3 Degree Selection

Table 3: SH parameter count by degree.

Max Degree L	Coeffs/channel	Total (RGB)	Effect
0	1	3	Constant diffuse
1	4	12	Linear shading
2	9	27	Quadratic specular
3	16	48	Sharp specular (used in 3DGS)

The original 3DGS paper uses $L = 3$. Training starts at $L = 0$ and unlocks one degree every 1000 iterations, stabilising early optimisation by fitting low-frequency geometry first.

5 2D Projection and Jacobian Approximation

5.1 Overview

Each 3D Gaussian must be projected onto the image plane. The key result is that the projection of a 3D Gaussian under a perspective camera yields *approximately* a 2D Gaussian, following the **EWA splatting** framework [9].

5.2 Camera Model and Mean Projection

A world-space point maps to camera space as $\mathbf{x}_c = \mathbf{R}_{cw}\,\mathbf{x}_w + \mathbf{t}_{cw}$, then projects to image coordinates:

$$\pi(\mathbf{t}) = \begin{pmatrix} f_x\,t_x/t_z + c_x \\ f_y\,t_y/t_z + c_y \end{pmatrix} \tag{10}$$

The 2D projected mean is simply $\boldsymbol{\mu}'_i = \pi(\mathbf{R}_{cw}\,\boldsymbol{\mu}_i + \mathbf{t}_{cw})$.

5.3 Covariance Projection via Jacobian Approximation

Since π is nonlinear, a first-order Taylor expansion is used. The Jacobian of π at camera-space point $\mathbf{t}_i = (t_{i,x}, t_{i,y}, t_{i,z})$ is:

$$\mathbf{J}_i = \begin{pmatrix} f_x/t_{i,z} & 0 & -f_x\,t_{i,x}/t_{i,z}^2 \\ 0 & f_y/t_{i,z} & -f_y\,t_{i,y}/t_{i,z}^2 \end{pmatrix} \in \mathbb{R}^{2\times 3} \tag{11}$$

The **projected 2D covariance** is:

$$\boxed{\boldsymbol{\Sigma}'_i = \mathbf{J}_i\,\mathbf{R}_{cw}\,\boldsymbol{\Sigma}_i\,\mathbf{R}_{cw}^\top\,\mathbf{J}_i^\top \in \mathbb{R}^{2\times 2}} \tag{12}$$

The resulting 2D splat in screen space is:

$$\mathcal{G}_i'(\mathbf{u}) = \exp\big(-\tfrac{1}{2}(\mathbf{u}-\boldsymbol{\mu}_i')^\top(\boldsymbol{\Sigma}_i')^{-1}(\mathbf{u}-\boldsymbol{\mu}_i')\big)\,, \quad \mathbf{u}\in\mathbb{R}^2 \tag{13}$$

$(\boldsymbol{\Sigma}_i')^{-1}$ is computed in closed form since it is 2×2.

5.4 Bounding Box and Tile Culling

For each splat, a bounding box of half-extent $r = 3\sqrt{\lambda_{\max}(\boldsymbol{\Sigma}_i')}$ (the 3σ ellipse, capturing $> 99\%$ of the Gaussian's mass) determines which screen tiles overlap with the splat. Pixels outside this radius are discarded, reducing per-tile Gaussian counts significantly.

The Jacobian-based approach is chosen because it is accurate for typical scene scales and fully differentiable, enabling gradient flow through the projection.

6 Differentiable Rasterisation Pipeline and Tiling

6.1 Design Goals

The pipeline must be: (1) **real-time** (<10 ms/frame at 1080p), (2) **differentiable** (gradients back to all Gaussian parameters), and (3) **depth-correct** (front-to-back compositing). These are achieved via a **tile-based, sorting-based CUDA rasteriser**.

6.2 Pipeline Overview

Algorithm 1 3DGS Differentiable Rasterisation Pipeline

Require: Gaussian set $\mathcal{S}$, camera parameters, resolution $W \times H$
Ensure: Rendered image $\hat{I}$

```
Preprocessing (parallel over all N Gaussians): project means, compute 2D
covariances, evaluate SH colours, compute bounding boxes and depths.
Tile assignment: divide screen into 16 × 16 tiles; for each Gaussian, append a
64-bit key [τ_id ‖ z_i] for every overlapping tile.
GPU radix sort by key ⇒ depth order per tile (O(M), ∼ 10^9 keys/s).
for each tile τ in parallel (one CUDA block) do
    for each pixel u in τ do
        Accumulate colour via alpha blending (Algorithm 2).
    end for
end for
return Î
```

6.3 Tile Decomposition

The screen is split into 16×16 pixel tiles (256 threads/block, a multiple of the CUDA warp size). Pixels in the same tile share most Gaussians, enabling cooperative shared-memory loading.

6.4 CUDA Rendering and Early Termination

Each CUDA block loads batches of 128 Gaussians into shared memory and each thread accumulates its pixel colour independently. A thread maintains accumulated transmittance T; when $T < \epsilon = 10^{-4}$ the pixel is *saturated* and the thread exits, providing significant speed-ups for occluded pixels.

6.5 Backward Pass

A custom CUDA kernel re-traces the forward pass in *reverse tile order* (back-to-front), accumulating gradients $\partial\mathcal{L}/\partial\boldsymbol{\mu}'_i$, $\partial\mathcal{L}/\partial\boldsymbol{\Sigma}'_i$, $\partial\mathcal{L}/\partial o_i$, $\partial\mathcal{L}/\partial\mathbf{c}_i$ via atomic operations. World-space gradients are then obtained by autodiff through the preprocessing pass. The overall pipeline scales as $O(N)$ in Gaussians and $O(WH)$ in pixels.

7 Alpha Blending and Accumulation Rendering

7.1 Alpha Compositing

Given N colour contributions $\{c_i\}$ with opacities $\{\alpha_i\}$ sorted front-to-back, the final pixel colour is:

$$\boxed{C = \sum_{i=1}^{N} c_i\,\alpha_i\,T_i}, \qquad T_i = \prod_{j=1}^{i-1}(1-\alpha_j), \quad T_1 = 1 \tag{14}$$

T_i is the transmittance reaching layer i; $\alpha_i T_i$ is its effective contribution.

7.2 Per-Gaussian Opacity at a Pixel

The opacity of Gaussian $\mathcal{G}_i$ at pixel $\mathbf{u}$ combines the learned opacity o_i and the 2D kernel response:

$$\boxed{\alpha_i(\mathbf{u}) = o_i \cdot \exp\!\big(-\tfrac{1}{2}(\mathbf{u}-\boldsymbol{\mu}'_i)^\top(\boldsymbol{\Sigma}'_i)^{-1}(\mathbf{u}-\boldsymbol{\mu}'_i)\big)} \tag{15}$$

7.3 Incremental Accumulation

Algorithm 2 Alpha Blending per Pixel (front-to-back)

Require: Sorted Gaussians $\{i_1, \ldots, i_K\}$ covering pixel $\mathbf{u}$
Ensure: Pixel colour $C(\mathbf{u})$

```
C ← 0,   T ← 1
for k = 1, ..., K do
    α ← α_{i_k}(u)
    if α < ε_min then continue
    end if
    C ← C + c_{i_k} · α · T;   T ← T · (1 − α)
    if T < ε_stop then break
    end if
end for
return C + T · C_bg
```

$\epsilon_{\text{min}} = \epsilon_{\text{stop}} = 10^{-4}$. Depth and opacity maps are obtained by the same accumulation replacing $\mathbf{c}_i$ with z_i or 1:

$$D(\mathbf{u}) = \sum_i z_i \, \alpha_i \, T_i, \qquad A(\mathbf{u}) = 1 - T_{\text{final}} \tag{16}$$

7.4 Connection to NeRF Volume Rendering

The 3DGS formula is the discrete equivalent of the NeRF volume integral. Identifying $\alpha_i = 1 - e^{-\sigma_i \delta_i}$ and $T_i = \prod_{j<i} e^{-\sigma_j \delta_j}$ recovers Eq. (14) exactly. The two methods differ only in *how* α_i is produced, not in the compositing formula.

8 Optimisation Mechanisms: Densification, Pruning, and Opacity

8.1 Training Objective

All parameters are optimised end-to-end by minimising:

$$\boxed{\mathcal{L} = (1 - \lambda)\, \mathcal{L}_1 + \lambda\, \mathcal{L}_{\text{D-SSIM}}}, \quad \lambda = 0.2 \tag{17}$$

$\mathcal{L}_1$ ensures per-pixel accuracy; $\mathcal{L}_{\text{D-SSIM}}$ preserves structural detail by penalising blurry renderings.

8.2 Optimiser

Adam with per-group learning rates (position: 1.6×10^{-4} decaying to 1.6×10^{-6}; scale: 5×10^{-3}; rotation: 10^{-3}; opacity: 5×10^{-2}; SH degree 0: 2.5×10^{-3}; SH 1–3: 2.5×10^{-4}).

8.3 Adaptive Density Control

Every 100 iterations (from iter. 500 to 15 000), the mean position gradient

$$\bar{\nabla}_i = \tfrac{1}{K} \sum_{k=1}^{K} \|\nabla_{\boldsymbol{\mu}_i'} \mathcal{L}\| \tag{18}$$

drives two operations when $\bar{\nabla}_i > \tau_{\text{grad}} = 2 \times 10^{-4}$:

- **Clone** (small Gaussian, $\max s_k < \tau_{\text{size}}$): create an exact copy; both are optimised independently. The region needs more primitives.
- **Split** (large Gaussian, $\max s_k \geq \tau_{\text{size}}$): replace with two Gaussians at scale $s_k/1.6$, positions sampled from $\mathcal{N}(\boldsymbol{\mu}_i, \boldsymbol{\Sigma}_i)$. The primitive covers too large an area.

8.4 Pruning

At the same frequency, Gaussians are removed if: (1) opacity $o_i < \epsilon_\alpha = 0.005$, (2) world-space size exceeds τ_w, or (3) projected 2D radius exceeds τ_v (removes "floaters").

8.5 Periodic Opacity Reset

Every 3000 iterations all opacities are reset to $\alpha_{\min} \approx 0.01$. Gaussians that contribute to the scene recover high opacity quickly; redundant ones do not and are pruned at the next pruning step. This prevents ghost Gaussians from accumulating throughout training.

8.6 Training Schedule and Convergence

Algorithm 3 3DGS Training Schedule (30 000 iterations)

Require: Images $\{I_k\}$, SfM point cloud $\mathcal{P}$

```
Initialise small isotropic Gaussians from P
for t = 1, ..., 30000 do
    Render Î; compute L; Adam step
    if 500 ≤ t ≤ 15000 and t mod 100 = 0 then
        Clone/split high-gradient Gaussians; prune low-opacity/large ones
    end if
    if t mod 3000 = 0 then Reset opacities to α_min
    end if
    if t mod 1000 = 0 and t ≤ 3000 then Unlock next SH degree
    end if
end for
```

A typical run starts with $\sim 10^4$–10^5 Gaussians, peaks at $\sim 3 \times 10^6$–6×10^6, and stabilises at $\sim 10^6$–4×10^6. Training takes 20–45 min on an RTX 3090/4090; at convergence rendering exceeds 90 fps at 1920×1080.

References

[1] Yuepeng Chen, Yifan Zhang, Heng Li, Sai Bi, Zexiang Xu, and Hao Su. NeRF-GS: NeRF-guided Gaussian splatting for physics-based material editing. In *arXiv preprint arXiv:2312.xxxx*, 2024.

[2] Robin Green. *Spherical Harmonic Lighting: The Gritty Details*. Sony Computer Entertainment America, 2003.

[3] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3D Gaussian Splatting for real-time radiance field rendering. In *ACM SIGGRAPH 2023 Conference Proceedings*. ACM, 2023.

[4] Ben Mildenhall, Pratul P. Srinivasan, Matthew Tancik, Jonathan T. Barron, Ravi Ramamoorthi, and Ren Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision (ECCV)*, pages 405–421. Springer, 2020.

[5] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM Transactions on Graphics*, 41(4):1–15, 2022.

[6] Johannes Lutz Schönberger and Jan-Michael Frahm. Structure-from-motion revisited. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4104–4113, 2016.

[7] Dor Verbin, Peter Hedman, Ben Mildenhall, Todd Zickler, Jonathan T. Barron, and Pratul P. Srinivasan. Ref-NeRF: Structured view-dependent appearance for neural radiance fields. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 5481–5490, 2022.

[8] Zhou Wang, Alan C. Bovik, Hamid R. Sheikh, and Eero P. Simoncelli. Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612, 2004.

[9] Matthias Zwicker, Hanspeter Pfister, Jeroen van Baar, and Markus Gross. EWA splatting. *IEEE Transactions on Visualization and Computer Graphics*, 8(3):223–238, 2002.